

Lecture 3

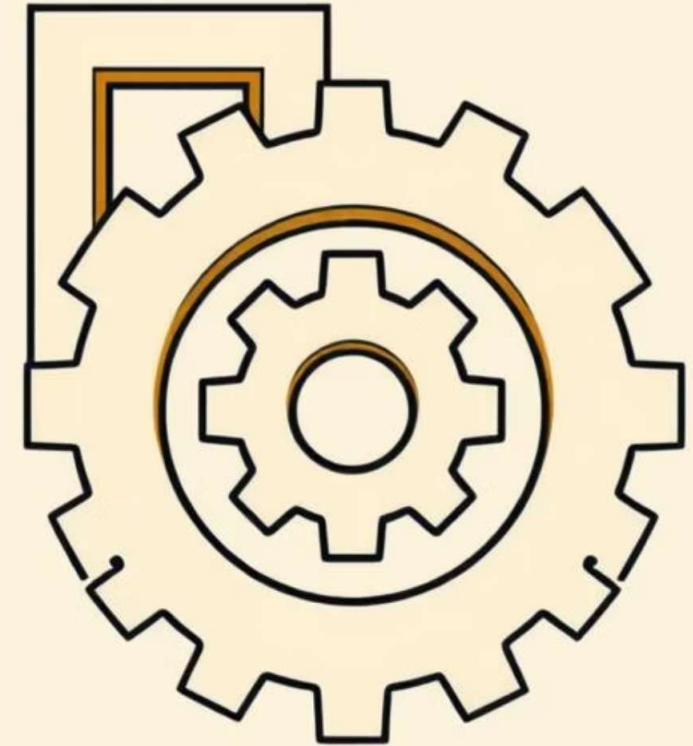
Structural Design Pattern, The Adapter

Bridging Incompatible Interfaces for Seamless Collaboration

Learning Objectives

- Understand the Role and Structure of the Adapter Pattern
 - Learn how the Adapter pattern bridges incompatible interfaces by introducing a middle layer that translates between them.
 - Identify key components: Target Interface, Adaptee, Adapter, and Client.
- Differentiate Between Adapter Implementations
 - Explore various forms of the Adapter pattern: Class Adapter (inheritance-based), Object Adapter (composition-based).
 - Evaluate which implementation suits different programming languages and system requirements.

Senior Lecturer – Almas Ospanov

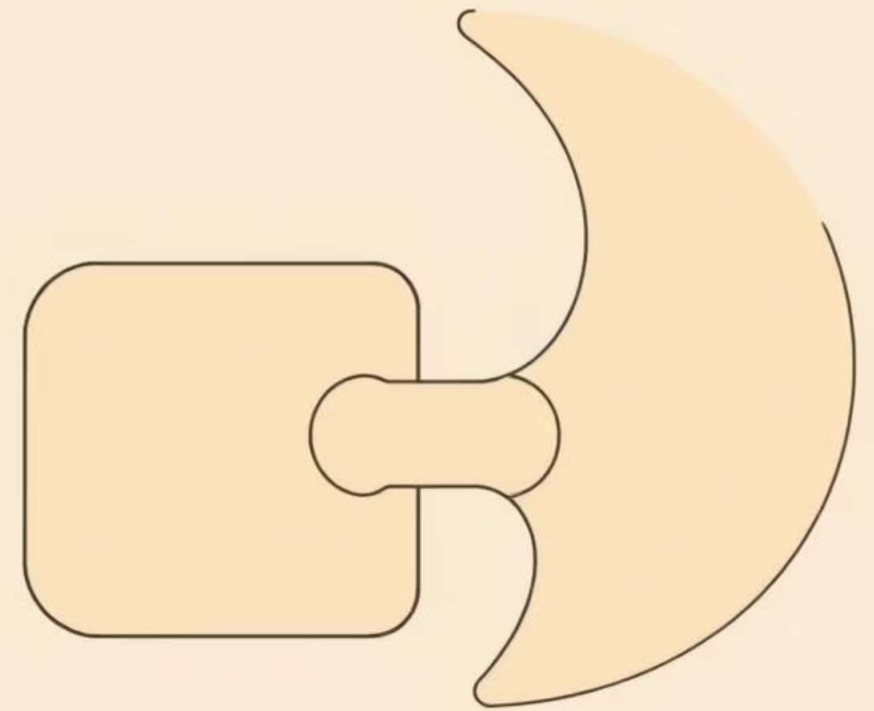


What is the Adapter Pattern?

In the world of software design, we often encounter situations where existing components don't quite fit together. **The Adapter pattern** is a powerful tool that allows incompatible interfaces to work seamlessly, enabling us to adapt designs and simplify complex systems.

The **Adapter pattern** is a structural design pattern that allows objects with incompatible interfaces to collaborate. It acts as a translator, converting the interface of one class into another interface that clients expect. This enables classes that couldn't otherwise work together due to interface mismatches to function harmoniously.

It's often referred to as a "Wrapper" because it wraps one of the objects to hide the complexity of the conversion happening behind the scenes. The wrapped object isn't even aware of the adapter's presence.





Here's how it works: The adapter receives calls from the client via a compatible interface. It then translates these requests into the format and order that the target object expects, passing the call along. This allows seamless communication between previously incompatible components.

- The Adapter pattern is a powerful tool for integrating disparate systems and promoting code reuse.
- It acts as a crucial bridge, making incompatible parts compatible and allowing objects with different interfaces to collaborate seamlessly.
-
- It's essential for working with legacy code or third-party libraries.

Introducing the Adapter (2/2)

To bridge the gap between incompatible interfaces, you write a class that adapts the new vendor interface into the one you're expecting. This class is the Adapter.

The Middleman

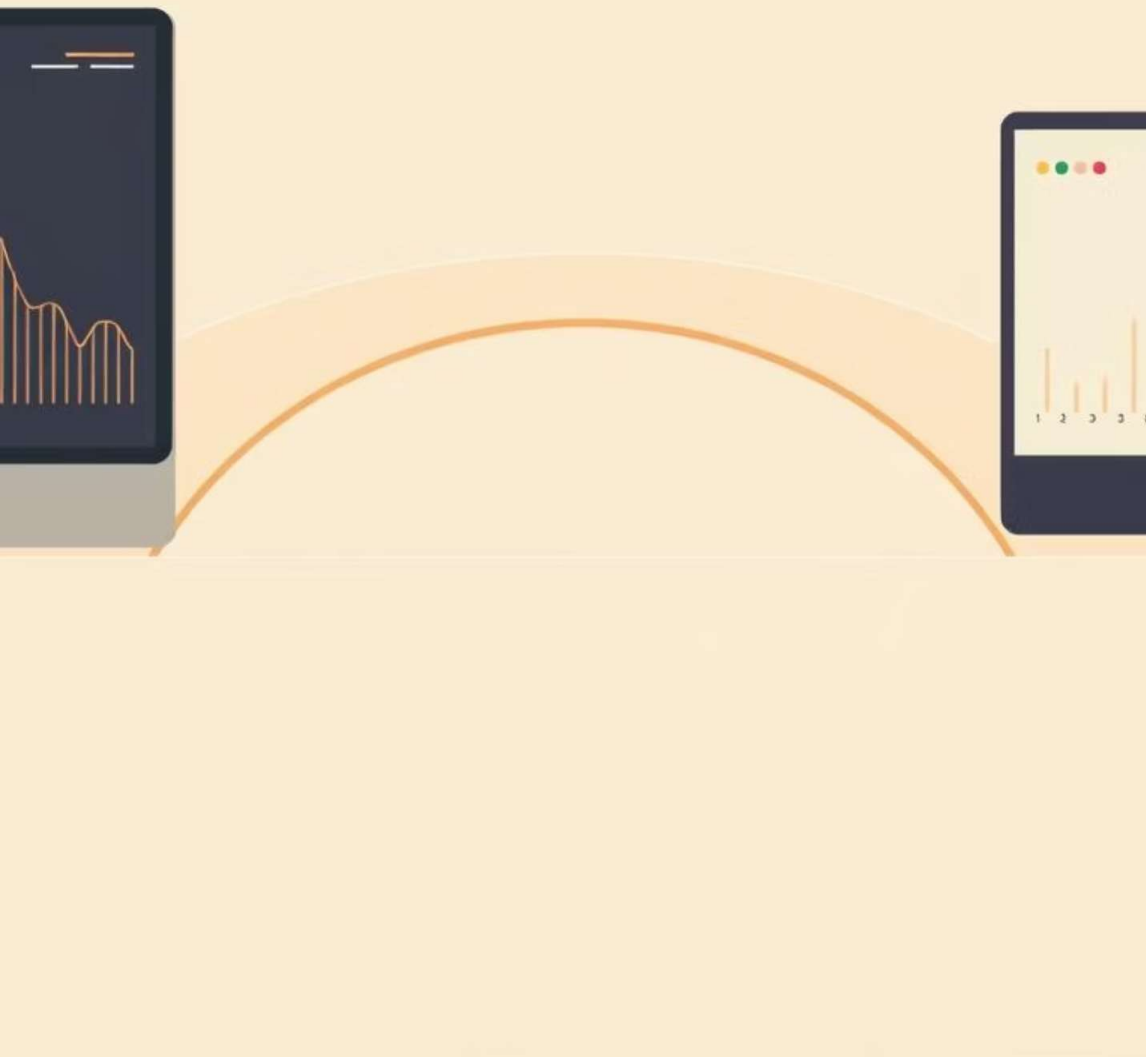
The adapter acts as an intermediary, receiving requests from the client.

Translating Requests

It converts these requests into calls that make sense to the vendor classes (the adaptee).

Seamless Integration

The client receives the results without ever knowing an adapter performed the translation.



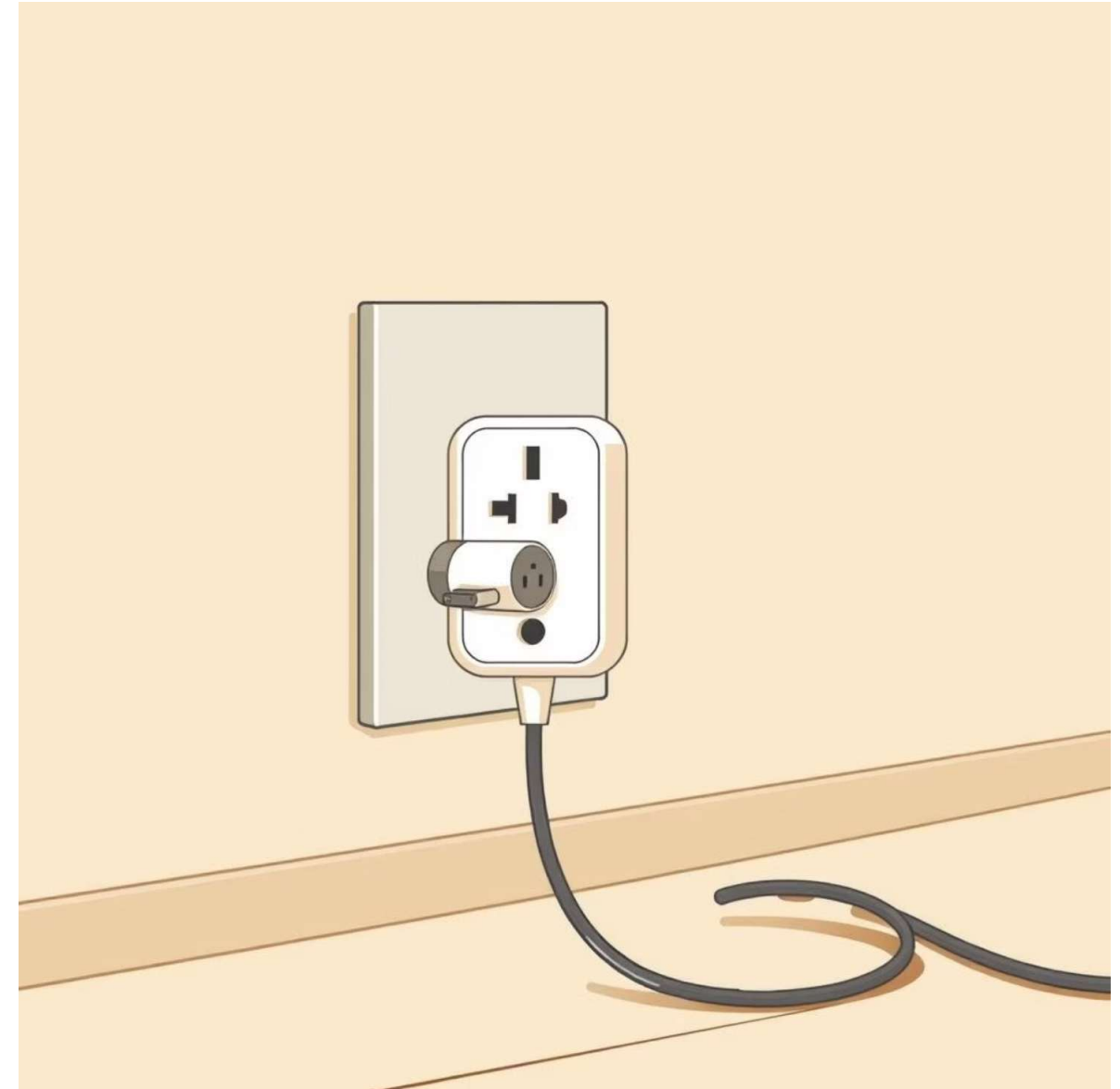
Real-World Analogy: Power Plug Adapter

Consider traveling from the US to Europe. Your US power plug won't fit a German wall socket due to different electrical standards. This is a classic example of incompatible interfaces.

A power plug adapter solves this problem. It has an American-style socket on one side and a European-style plug on the other. It doesn't change your device, but it allows your device to connect to an incompatible power source, just as a software adapter enables incompatible code interfaces to connect.

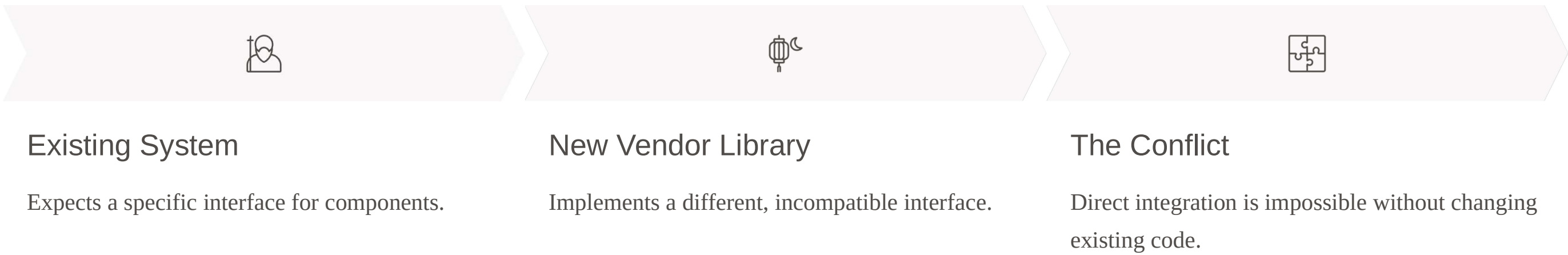
More examples how adapters allow systems to work together without modifying their core logic—perfect for maintaining clean architecture in evolving environments:

- **Memory Card Reader** - A laptop may not support SD cards directly, but a USB card reader acts as an adapter. It translates SD card interface into USB, allowing the laptop to read the data.
- **Legacy System Integration** - A modern app expects JSON data, but a legacy system outputs XML. An adapter class converts XML to JSON so the new app can consume it without changing its internal logic.
- **Payment Gateway Wrappers** - An e-commerce platform expects a `processPayment()` method, but Stripe uses `charge()`. Just write a `StripeAdapter` that wraps Stripe's API and exposes `processPayment()` to the app.



The Challenge: Incompatible Interfaces

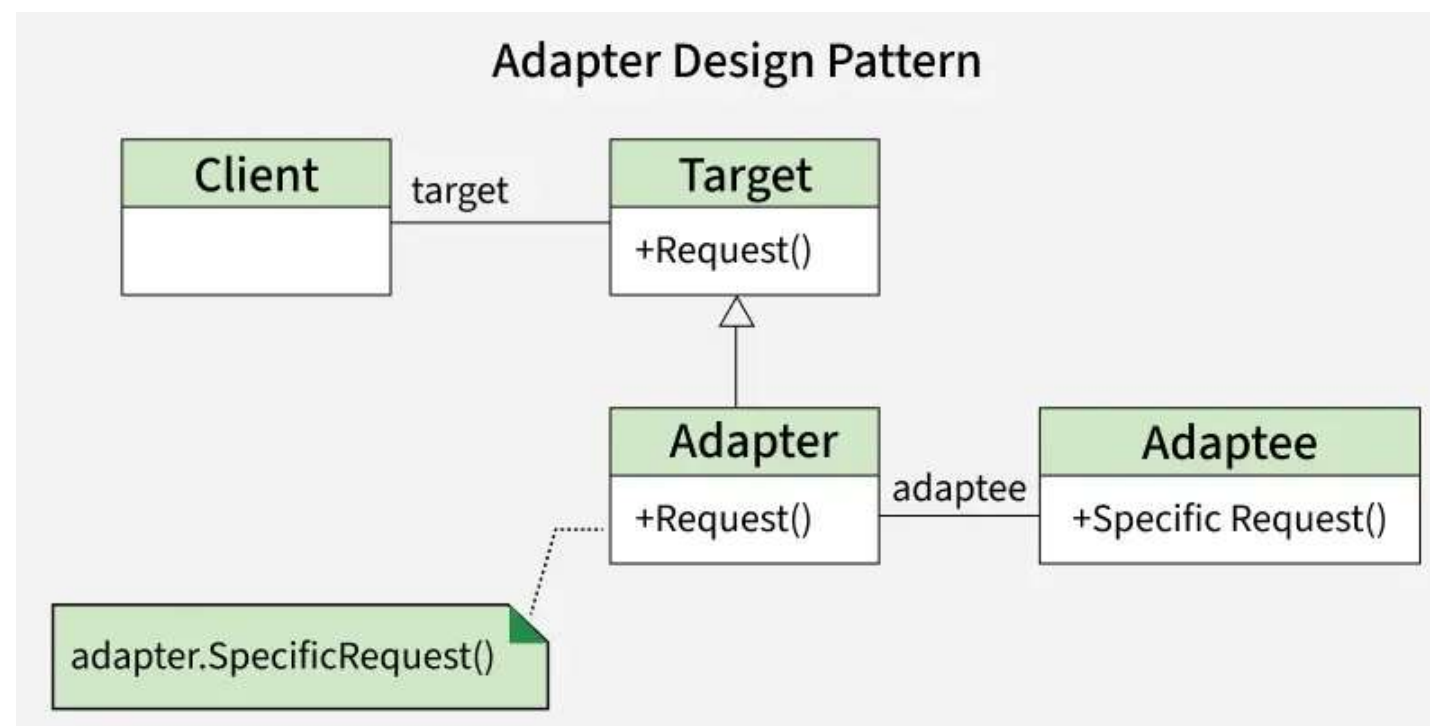
Imagine you have an existing software system, and you need to integrate a new vendor's class library. The problem? The new vendor designed their interfaces differently from your existing system or previous vendors.



You can't change your existing code, and you can't change the vendor's code. This is where the Adapter Pattern provides a clean solution.

Understanding the Adapter pattern (1/3)

- **Explanation of the diagram:**
- Client uses a Target interface by calling Request().
- Adaptee already has helpful functionality, but its method (SpecificRequest()) doesn't match the Target interface.
- Adapter acts as a bridge: it implements the Target interface (Request()), although internally, it calls the Adaptee's SpecificRequest().
- This approach allows the Client to use the Adaptee without changing its code.



Understanding the Adapter pattern (2/3)

Components of Adapter Pattern's components:

- **Target Interface:** The interface anticipated by the **client**, defining the operations it can use.
- **Adaptee:** The existing class with an incompatible interface that needs an integration.
- **Adapter:** Implements the target interface and uses the **adaptee** internally, acting as a bridge.
- **Client:** Uses the **target interface**, where it's completely unconscious of the **adapter** or **adaptee** details.

How Adapter Design Pattern works?

1. **Client** starts a request by calling a method on the **adapter** through the **target interface**.
2. **Adapter** represents the **client's** request into a format that the **adaptee** can understand by using its interface.
3. **Adaptee** does the actual job based on the re-translated request from the **adapter**.
4. **Client** receives the results of the call, remaining unaware of the adapter's existence or the specific details described in the **adaptee**.

Understanding the Adapter pattern (3/3)

☑ When to Use the Adapter Design Pattern

- Facilitates interaction between systems or components that don't naturally align in terms of interface or structure.
- Allows you to leverage existing classes or libraries without modifying their internal code.
- Eases the process of integrating new modules while maintaining architectural flexibility.
- Consolidates interface adjustments in one place, improving maintainability and reducing risk.

⊗ When to Avoid the Adapter Design Pattern

- If your system is simple and all components already work together seamlessly, introducing an adapter adds unnecessary complexity.
- In performance-critical applications, the minor overhead introduced by adapters may be undesirable.
- When interface compatibility isn't an issue, using an adapter offers no real benefit.
- For short-term or throwaway projects, the effort to implement adapters may outweigh their value.

Variants of the Adapter Design Pattern (1/2)

The Adapter pattern can be implemented in several distinct ways, depending on the programming language and the integration scenario.

1. Class Adapter (Based on Inheritance)

- This method involves creating an adapter that inherits from both the target interface (expected by the client) and the adaptee (the existing class to be adapted).
- It's primarily used in languages like C++ that support multiple inheritance.
- In contrast, languages such as Java and C#—which restrict multiple inheritance—rarely apply this technique.

2. Object Adapter (Based on Composition)

- Instead of inheritance, this approach uses composition: the adapter contains a reference to the adaptee and implements the target interface.
- It offers greater flexibility, allowing one adapter to work with various adaptee instances, and avoids the complexity associated with inheritance.
- This is the preferred method in languages like Java and C#.

Variants of the Adapter Design Pattern (2/2)

3. Two-way Adapter

- This adapter serves a dual role—it can act as both the target and the adaptee, depending on which interface is being accessed.
- It's especially valuable when two systems need to interoperate and adapt to each other reciprocally.

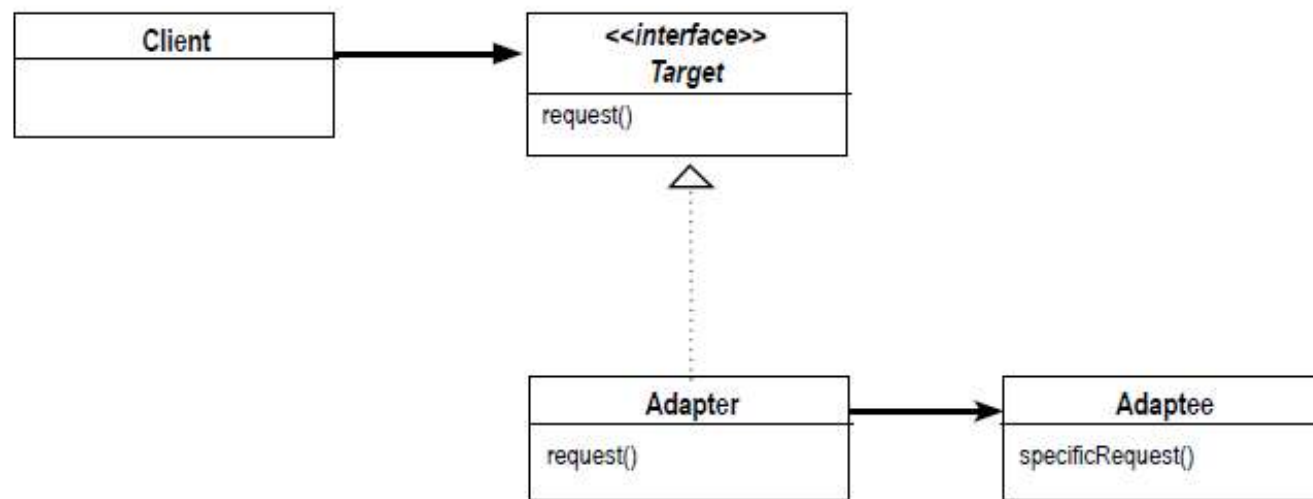
4. Interface Adapter (Also Known as Default Adapter)

- When only a subset of an interface's methods is needed, an interface adapter can be used to provide default implementations for the unused methods.
- This is particularly helpful when dealing with interfaces that define many methods.
- Languages like Java support this pattern through abstract classes or default methods in interfaces, simplifying the implementation.

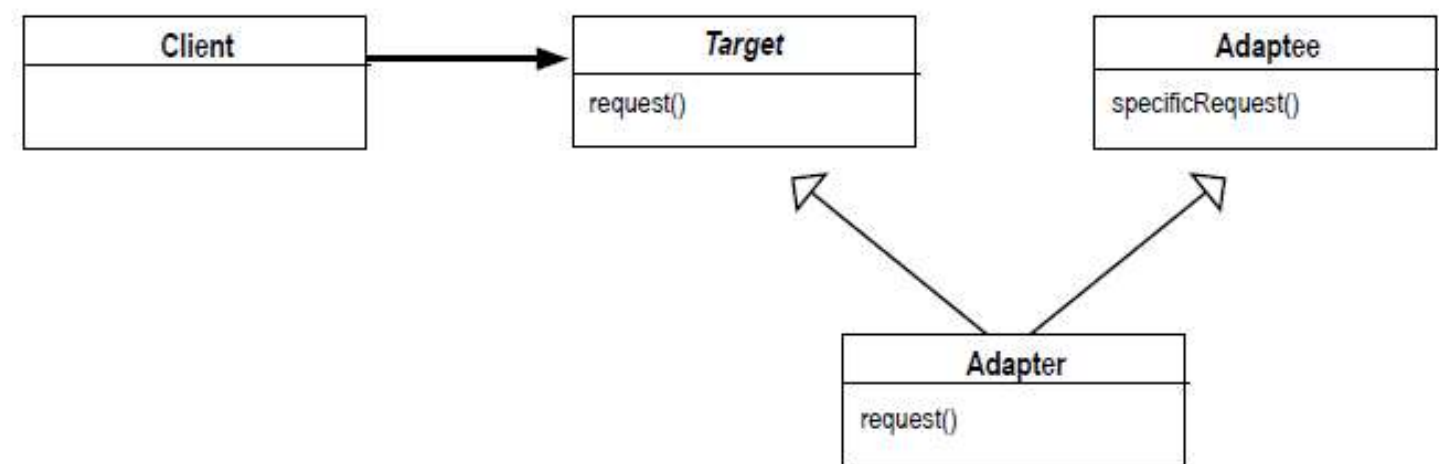
Object Adapter vs. Class Adapter

Let's focus on two main forms of the Adapter Pattern

Object Adapter



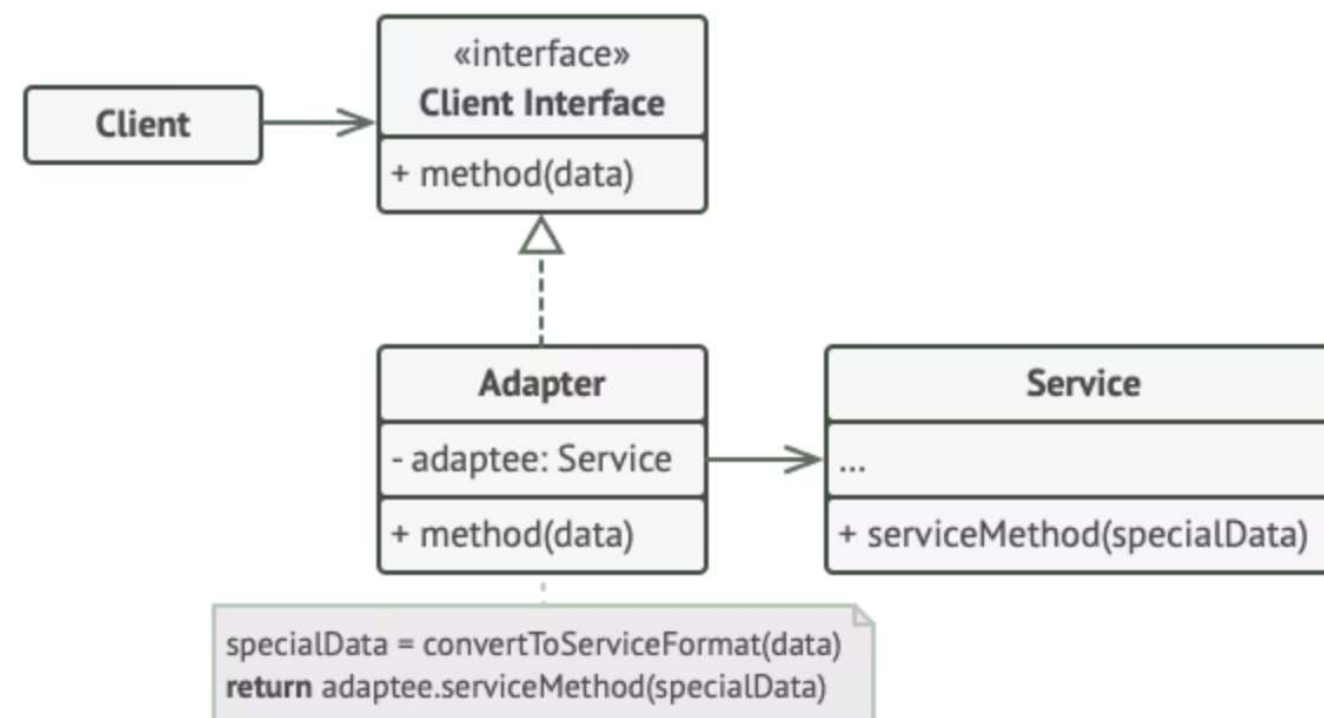
Class Adapter



Structure: Object Adapter

The Object Adapter implementation uses the principle of **object composition**. The adapter implements the interface that the client expects, while internally holding a reference to the service object with the incompatible interface.

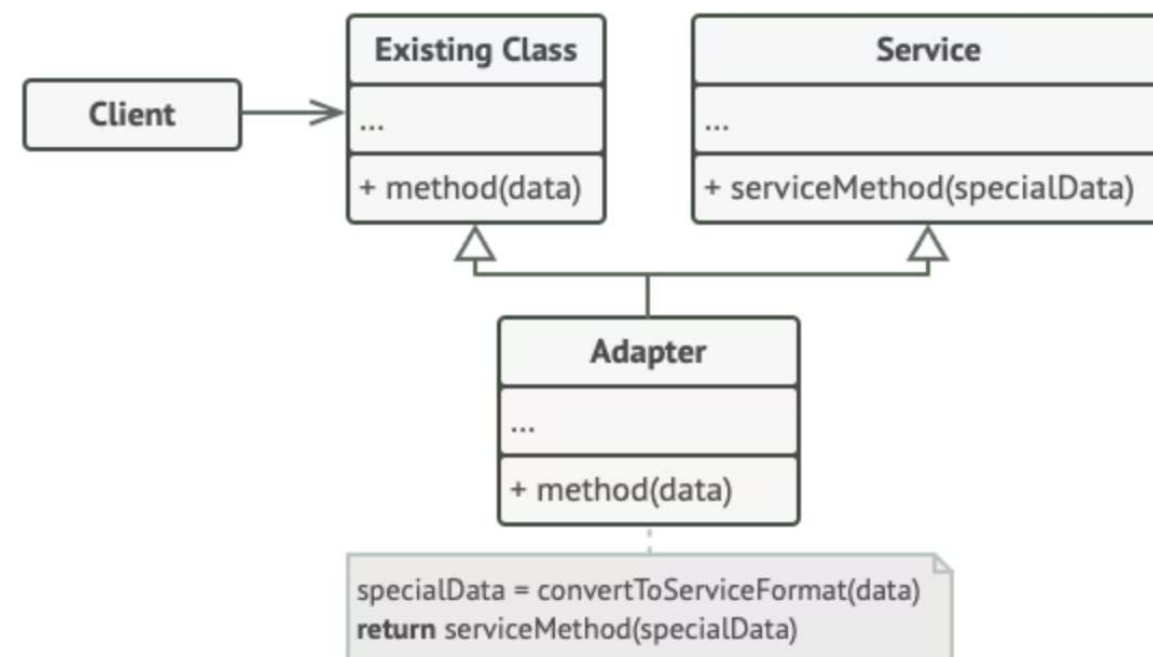
The client interacts with the adapter through the familiar client interface. The adapter then translates these calls and delegates the actual work to the wrapped service object. This approach offers flexibility and is widely supported across programming languages.



Structure: Class Adapter

The Class Adapter implementation utilizes **inheritance**. In this approach, the adapter class inherits from both the client's expected interface and the service's concrete class. This means the adapter directly gains the behaviors of both.

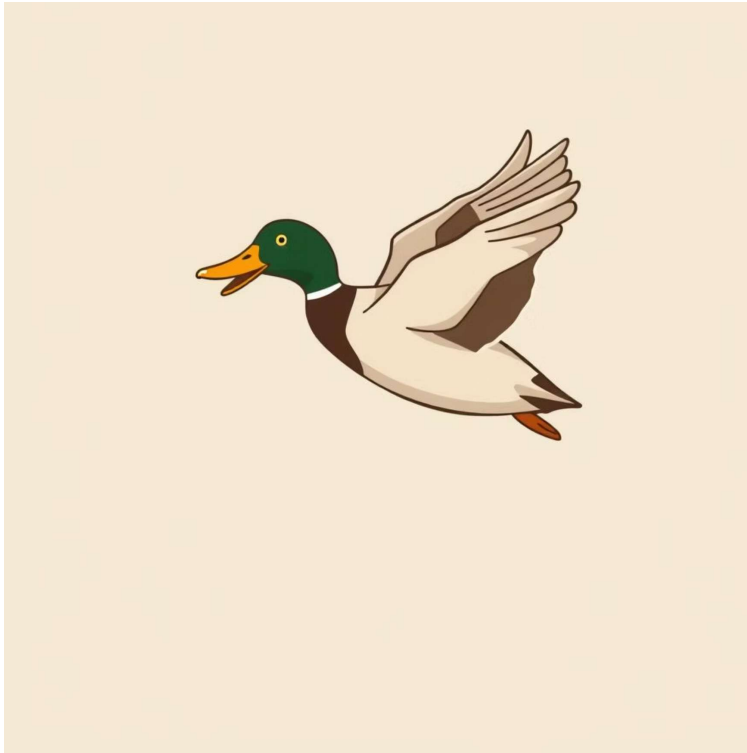
This pattern is only feasible in programming languages that support multiple inheritance, such as C++. The adaptation logic is embedded within the overridden methods of the adapter class, allowing it to be used directly in place of an existing client class.



Adapter in Action: The Duck and the Turkey

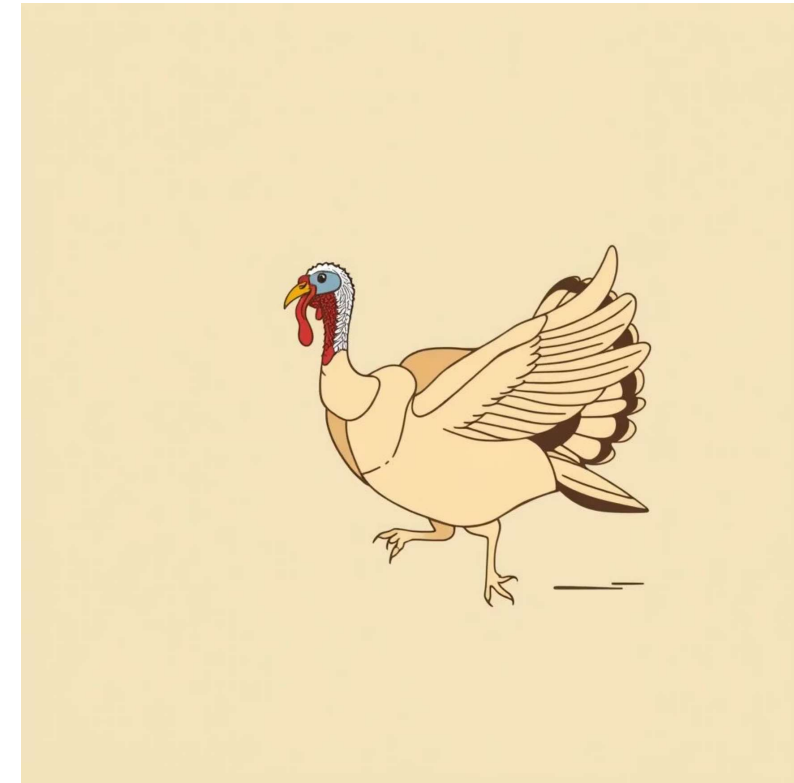
Let's illustrate the Adapter Pattern with a classic example: adapting a Turkey to act like a Duck. Our `Duck` interface defines `quack()` and `fly()`, while a `Turkey` has `gobble()` and a short `fly()`.

The Duck



Ducks quack and fly long distances.

The Turkey



Turkeys gobble and fly only short distances.

If you're short on Duck objects but have Turkeys, you can't use them directly due to their different interfaces. An Adapter is needed to make the Turkey appear as a Duck.

Implementing the **TurkeyAdapter**

The `TurkeyAdapter` implements the `Duck` interface, making it look like a Duck to the client. It holds a reference to the `Turkey` object it's adapting.

1

Implement Duck Interface

The adapter implements the `Duck` interface, which the client expects.

2

Reference Turkey

It takes a `Turkey` object in its constructor to adapt.

3

Translate Quack()

The `quack()` method calls the `turkey.gobble()` method.

4

Translate Fly()

The `fly()` method calls `turkey.fly()` five times to simulate a duck's longer flight.

This translation allows the client to interact with the Turkey as if it were a Duck, without knowing the underlying implementation details.

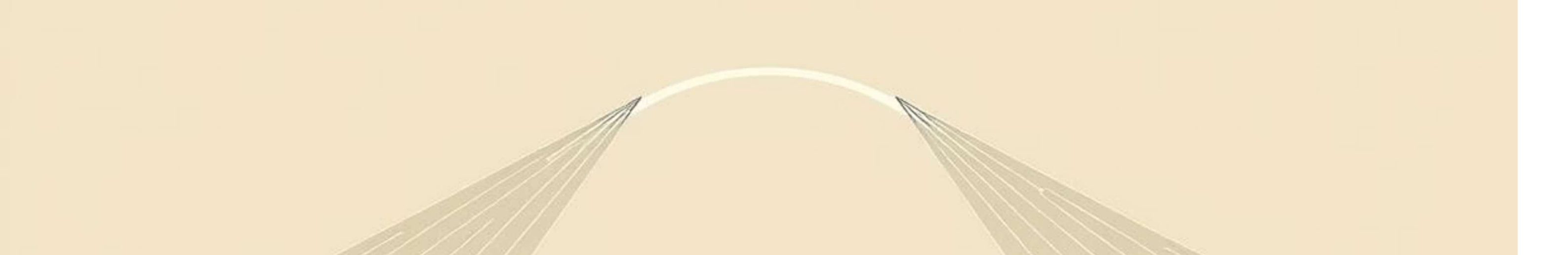
Test Driving the Adapter

When we test the `TurkeyAdapter`, the client code, which expects a `Duck` object, interacts with the adapted Turkey seamlessly. The `testDuck()` method calls `quack()` and `fly()` on the adapter.

```

The Turkey says...
    Gobble gobble
    I'm flying a short distance
The Duck says...
    Quack
    I'm flying
The TurkeyAdapter says...
    Gobble gobble
    I'm flying a short distance
    I'm flying a short distance
    I'm flying a short distance
    I'm flying a short distance
    I'm flying a short distance
```

The output shows the Turkey gobbling when `quack()` is called, and flying multiple times when `fly()` is invoked. The `testDuck()` method never knows it's interacting with a Turkey disguised as a Duck!



When to Apply the Adapter Pattern

Incompatible Interfaces

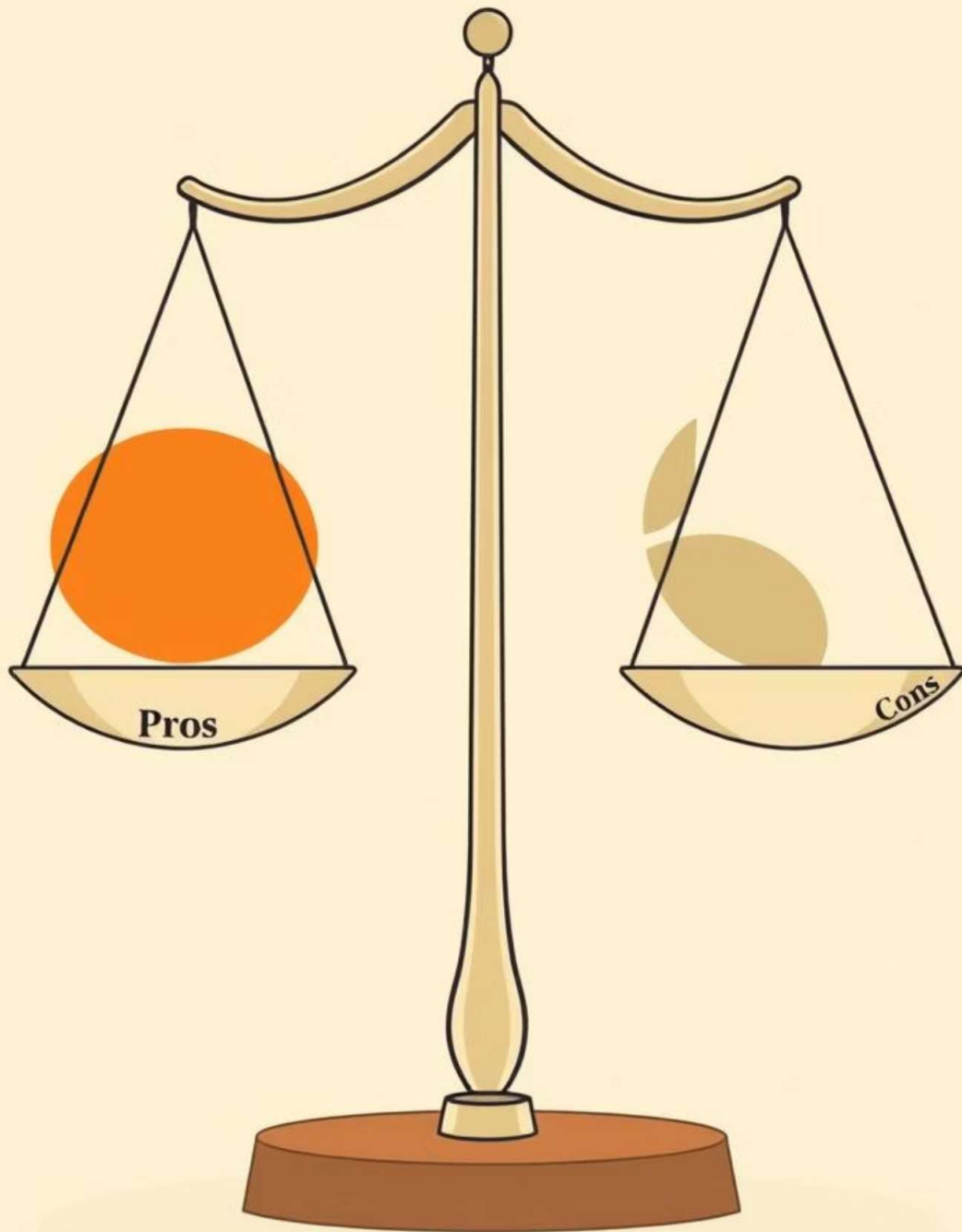
When you want to use an existing class, but its interface isn't compatible with the rest of your code.

Legacy or 3rd-Party Systems

To create a middle-layer translator between your code and a legacy system, a 3rd-party library, or any class with an unusual interface.

Reusing Subclasses

When reusing several existing subclasses that lack common functionality, wrap them in an adapter to dynamically add the missing features.



Advantages & Disadvantages

Pros

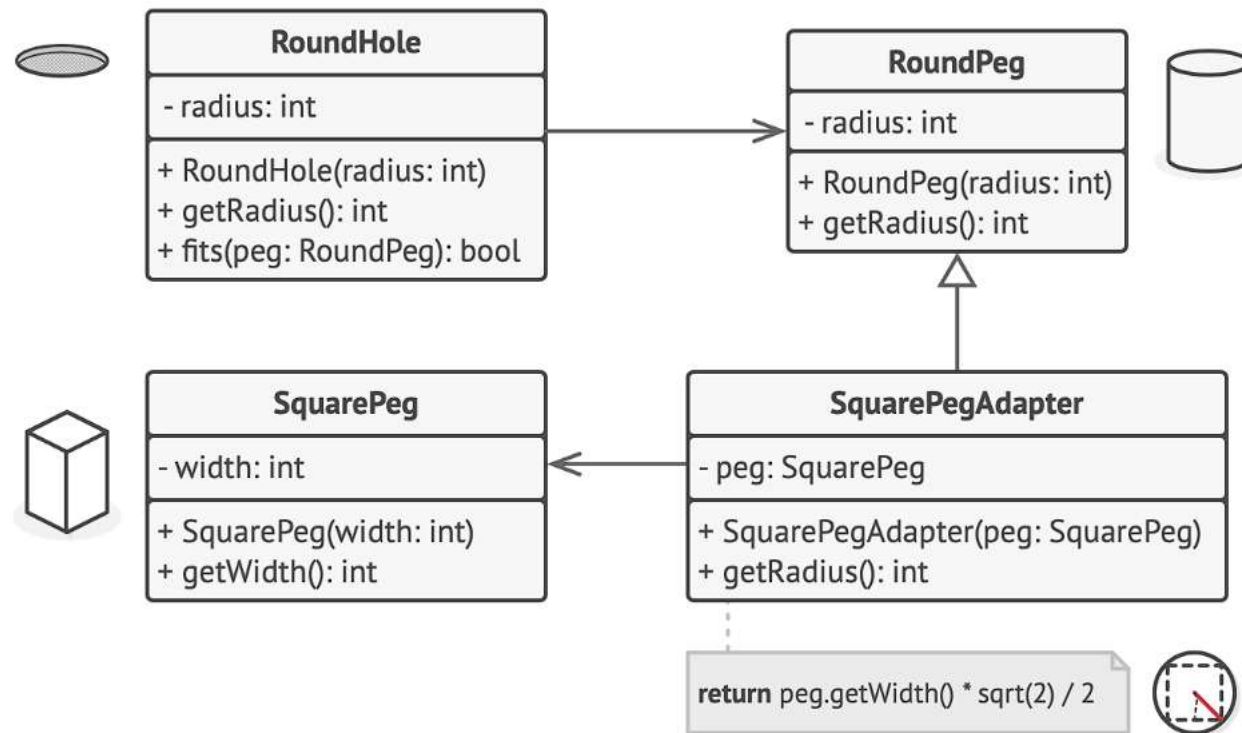
- **Single Responsibility Principle:** Separates interface/data conversion code from primary business logic.
- **Open/Closed Principle:** Allows introducing new types of adapters without modifying existing client code.
- **Code Reusability:** Enables using existing classes that were otherwise incompatible.

Cons

- **Increased Complexity:** Introduces new interfaces and classes, potentially making the codebase more complex.
- **Potential Over-engineering:** For simple cases, directly changing the service class might be a simpler solution.

Code example (1/2)

Pseudocode Example: Square Peg Adapter



Adapting square pegs to round holes.

Source: refactoring.guru

This illustration of the Adapter pattern draws from the well-known mismatch between square pegs and round holes—highlighting how incompatible components can be made to fit together through a bridging solution.

The Adapter pretends to be a round peg, with a radius is equal to a half of the square's diameter: the radius of the smallest circle that can accommodate the square peg.

Code example (2/2)

```
// Say you have two classes with compatible interfaces:  
// RoundHole and RoundPeg.
```

```
class RoundHole is  
  constructor RoundHole(radius) { ... }  
  method getRadius() is  
    // Return the radius of the hole.
```

```
  method fits(peg: RoundPeg) is  
    return this.getRadius() >= peg.getRadius()
```

```
class RoundPeg is  
  constructor RoundPeg(radius) { ... }
```

```
  method getRadius() is  
    // Return the radius of the peg.
```

```
// But there's an incompatible class: SquarePeg.  
class SquarePeg is
```

```
  constructor SquarePeg(width) { ... }
```

```
  method getWidth() is // Return the square peg width.
```

```
// An adapter class lets you fit square pegs into round holes.
```

```
// It extends the RoundPeg class to let the adapter objects act  
// as round pegs.
```

```
class SquarePegAdapter extends RoundPeg is
```

```
  // In reality, the adapter contains an instance of the  
  // SquarePeg class.
```

```
  private field peg: SquarePeg
```

```
  constructor SquarePegAdapter(peg: SquarePeg) is  
    this.peg = peg
```

```
  method getRadius() is
```

```
    // The adapter pretends that it's a round peg with a  
    // radius that could fit the square peg that the adapter  
    // actually wraps.  
    return peg.getWidth() * Math.sqrt(2) / 2
```

```
// Somewhere in client code.
```

```
hole = new RoundHole(5)  
rpeg = new RoundPeg(5)  
hole.fits(rpeg) // true  
small_sqpeg = new SquarePeg(5)  
large_sqpeg = new SquarePeg(10)  
hole.fits(small_sqpeg) // this won't compile (incompatible types)
```

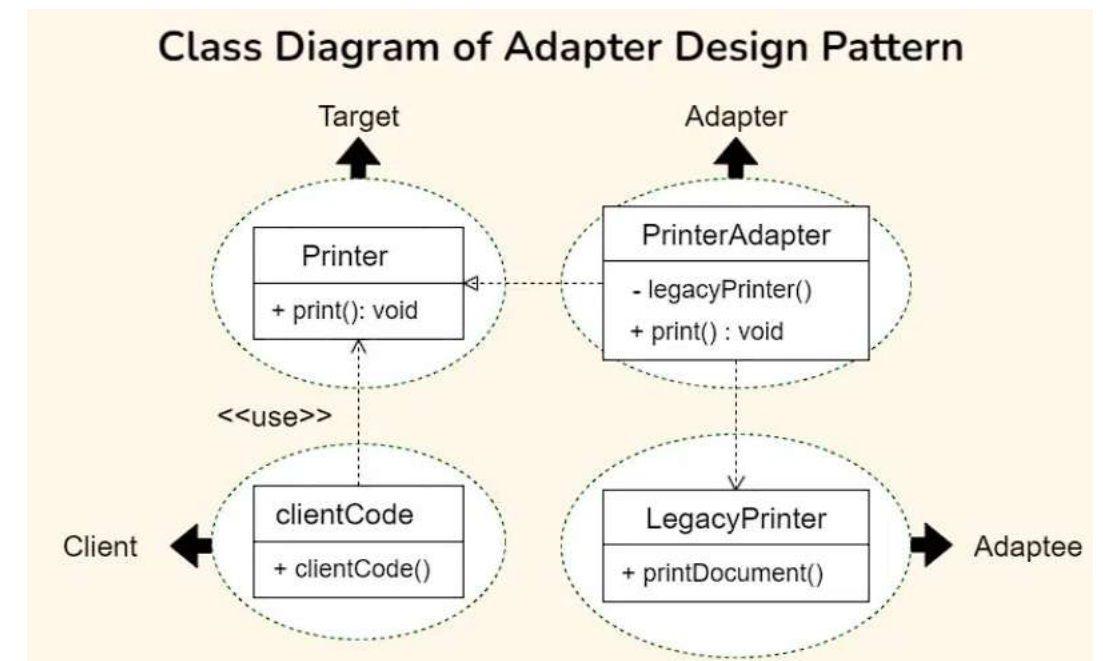
```
small_sqpeg_adapter = new SquarePegAdapter(small_sqpeg)  
large_sqpeg_adapter = new SquarePegAdapter(large_sqpeg)  
hole.fits(small_sqpeg_adapter) // true  
hole.fits(large_sqpeg_adapter) // false
```

Another example (1/2)

Problem statement: Imagine we're working with an older system that relies on a **LegacyPrinter** class featuring a method called **printDocument()**. Meanwhile, a newer system expects components to conform to a **Printer** interface that defines a method named **print()**. To bridge this mismatch and enable seamless integration, we'll apply **the Adapter design pattern**—creating a wrapper that translates between the legacy method and the expected interface.

Expected Output:

" Legacy Printer is printing a document. "



Source: [geekforgeeks.org](https://www.geekforgeeks.org/)

Another example (2/2)

/ Adapter Design Pattern Example Code */*

// Target Interface – the interface that the client code expects

```
interface Printer {  
    void print();  
}
```

// Adaptee – The existing class with an incompatible interface

```
class LegacyPrinter {  
    public void printDocument() {  
        System.out.println("Legacy Printer is printing a document.");  
    }  
}
```

*// Adapter (PrinterAdapter) – The class that adapts the
// LegacyPrinter to the Printer Interface*

```
class PrinterAdapter implements Printer {  
    private LegacyPrinter legacyPrinter = new LegacyPrinter();  
  
    @Override  
    public void print() {  
        legacyPrinter.printDocument();  
    }  
}
```

*// Client Code – The code that interacts with the
Printer Interface*

```
public class Client {  
    public static void clientCode(Printer printer) {  
        printer.print();  
    }  
  
    public static void main(String[] args) {  
        // Using the Adapter  
        PrinterAdapter adapter = new PrinterAdapter();  
        clientCode(adapter);  
    }  
}
```

Interactions with other patterns

- **Bridge pattern** is usually designed first which is letting to develop parts of an application independently of each other whereas **Adapter pattern** is used with an already existing app to allowing classes that weren't originally designed to work together to collaborate seamlessly.
- **Adapter pattern** supplies with a completely different interface for getting access into an existing object. But, with the **Decorator pattern** the interface either, at least, stays the same or can be extended. Additionally, **Decorator pattern** assists in recursive composition, which isn't possible when you use **Adapter pattern**.
- **Adapter pattern** lets to access existing objects through different interfaces, while **Proxy pattern** the interface stays unchanged. On the other hand, **Decorator pattern** lets to access the object via an enhanced interface.
- **Facade pattern** defines a new interface for existing objects, whereas **Adapter pattern** attends to make the existing interface usable. **Adapter pattern** wraps just one object, while **Facade pattern** deals with an entire subsystem of objects.
- **Bridge, State, Strategy** and at some point **Adapter patterns** have very similar structures. In fact, all of them are based on composition: delegating work to other objects. Nevertheless, all these patterns solve different problems.

Note: A pattern is not just a formula for structuring your code in a specific way. It, also, helps to communicate to other developers about the problem the pattern solves.

Key Takeaways:

Interface Conversion

Changes an interface into one a client expects.

Decoupling

Decouples a client from an incompatible class.

Composition/Inheritance

Achieved through object composition (most common) or multiple inheritance.

Flexibility

Allows existing classes to work together without modification.

The Adapter Pattern is a fundamental tool in object-oriented design, promoting re-usability and maintainability by resolving interface incompatibilities gracefully.